IA E LABORATORIO · PROGETTO · A.A. 2025/26

Dal labirinto al torneo

A* e IDA* in Prolog · calendario di un torneo Pokémon in ASP

Alfredo De Donno · matricola 810700

Modulo del Prof. G. L. Pozzato

ARCHITETTURA

Gli algoritmi non sanno nulla del dominio

Domini

`labirinto.pl`

`puzzle8.pl`

azioni ed euristica
+ istanze in test/

Interfaccia vista a lezione

```
iniziale(S)
finale(S)
applicabile(Az, S)
trasforma(Az, S, S1)
euristica(S, H)
```

Algoritmi

`astar.pl`

`idastar.pl`

un solo file per
entrambi i domini

Stessa separazione dominio / motore del planning: aggiungere un dominio non tocca gli algoritmi.

A*

Frontiera a heap, chiusi in un albero

```
astar_loop(Open, Chiusi, Rev, Costo) :-
  get_from_heap(Open, _, nodo(S,G,Cam), Open1),
  ( get_assoc(S, Chiusi, _)
  → astar_loop(Open1, Chiusi, Rev, Costo)
  ;  finale(S)
  → Rev = Cam, Costo = G
  ;  put_assoc(S, Chiusi, G, Chiusi1),
     findall(Az-S1, ..., Figli),
     aggiungi_figli(...),
     astar_loop(Open2, Chiusi1, Rev, Costo) ).
```

library(heaps): estrarre il minimo f costa O(log n), non O(n)

library(assoc): "è già chiuso?" in O(log n) invece di member/2

Cancellazione pigra: le copie vecchie di uno stato si scartano quando escono

Euristiche consistenti: un nodo chiuso ha già il g ottimo, nessuna riapertura

IDA*

Profondità limitata su f, memoria lineare

```
dfs_soglia(S, G, Soglia, Cam, Az, C) :-
  euristica(S, H), F is G + H,
  ( F > Soglia
  → aggiorna_minimo(F), fail
  ;  finale(S) → Az = [], C = G
  ;  applicabile(A, S), trasforma(A, S, S1),
     \+ memberchk(S1, Cam),
     ...
     dfs_soglia(S1, G1, Soglia, [S1|Cam], ...) ).
```

Prima soglia = h(s0); poi il **minimo f tagliato**

nb_setval conserva quel minimo attraverso il backtracking

Cicli controllati solo sul cammino corrente: niente lista dei chiusi

Con h ammissibile la prima soluzione è **ottima**, come A*

DOMINI ED EURISTICHE

Tutte ammissibili e consistenti

Labirinto con più uscite

h = distanza di Manhattan dall'**uscita più vicina**

Una mossa cambia ogni distanza al più di 1, quindi anche il minimo: h è consistente.

Limite: un'uscita vicina ma murata attira la ricerca nel vicolo cieco.

Puzzle dell'8

Manhattan ≥ **tessere fuori posto** ≥ h = 0

Verificato su tutti i 181.440 stati: in media h vale il 64% della distanza vera con Manhattan, il 32% con fuori posto.

risolvibile/1: parità delle inversioni. Metà delle configurazioni non ha soluzione.

METODO SPERIMENTALE

Stesse istanze, ottimo verificato altrove

6

labirinti nostri, da 10×10 a 30×30

12

puzzle da 4 a 31 mosse (il massimo)

72

esecuzioni, ognuna in un processo swipl separato

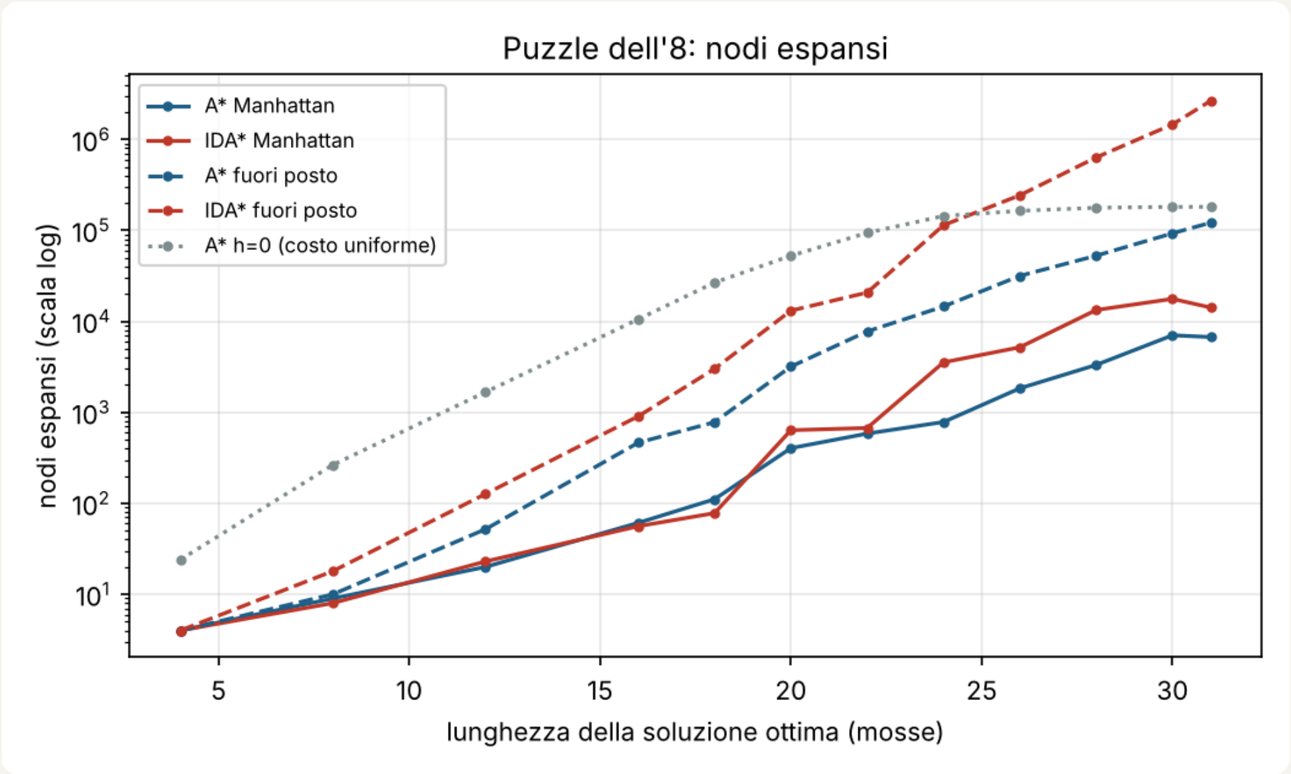
72/72

costi ottimi, confrontati con una BFS in Python

Misure: nodi espansi, stati in memoria, tempo CPU. Timeout di 60 s. Due labirinti "cattivi" apposta: un'uscita vicina ma recintata e nessuna uscita raggiungibile.

RISULTATI · PUZZLE DELL'8

L'euristica conta più dell'algoritmo



p31: **6.726** nodi con Manhattan, **121.530** con fuori posto, **181.439** con h = 0

IDA* espande 1–4 volte più nodi, ma i **tempi sono pari** (0,36 s contro 0,38 s)

Memoria su p31: **32** stati per IDA*, **10.394** per A*

RISULTATI · LABIRINTI

Senza lista dei chiusi, IDA* crolla

Labirinto	Ottimo	A* nodi	IDA* nodi
lab1 piccolo 10×10	21	45	249
lab2 grande 20×40	76	397	35.625
lab3 trappola	26	125	timeout
lab4 senza uscita	nessuna	24	374.646
lab5 aperto 20×20	27	54	2.059
lab6 aperto 30×30	39	79	23.127

Trasposizioni: in una griglia la stessa cella si raggiunge per moltissimi cammini, e IDA* li rivisita tutti

Trappola: l'uscita recintata attira l'euristica. A* esce in 125 nodi, IDA* va in timeout dopo oltre 4 milioni

Nessuna uscita: A* lo dimostra in 24 nodi

ATTIVITÀ 2 · IL MODELLO ASP

Generate, test, optimize

```
% generate: un BYE se dispari, mai a chi l'ha già avuto
1 { bye(P) : giocatore(P), not bye_avuto(P) } 1 :- dispari.
% generate: solo coppie mai affrontate; P < Q evita (a,b)/(b,a)
{ incontro(P,Q) : giocatore(P), giocatore(Q), P < Q, not gia_affrontati(P,Q) }.
% test: ognuno impegnato esattamente una volta
:- giocatore(P), not impegnato(P).
:- giocatore(P), 2 { incontro(P,Q):giocatore(Q) ; incontro(Q,P):giocatore(Q) ; bye(P) }.
% optimize, per priorità
#minimize { 1@3,P,Q : misto(P,Q) }.          % pari punteggio
#minimize { D@2,P,Q : incontro(P,Q), ..., D = |NP-NQ| }.
#minimize { N@1,P : bye(P), pt(P,N) }.     % BYE ai più bassi
```

Input per turno: giocatore/1, giocato/2, bye_avuto/1, punti/2. I nuovi iscritti valgono 0 punti; i vincoli della consegna sono già nella fase generate.

ATTIVITÀ 2 · RISULTATI

L'esempio della consegna e oltre

Turno 2 dell'esempio

8 giocatori, 3 a quota 2 punti: un incontro misto è inevitabile

39 abbinamenti ottimi, costo [1, 2]. Tra questi c'è quello proposto dalla consegna

Torneo simulato

5 turni, nuovi iscritti nei turni 2 e 4

Ogni turno è **verificato in Python**, indipendentemente da ASP. Con 32 giocatori: meno di 0,1 s a turno

Caso impossibile

4 giocatori: dopo 3 turni tutti si sono già affrontati

Turno 4 → **UNSATISFIABLE**: ASP lo dimostra invece di inventare un abbinamento

LA SCOPERTA DEL PROGETTO

Stessi ottimi, prestazioni opposte

Giocatori	#maximize pari	#minimize misti
12	0,14 s	0,000 s
14	3,4 s	0,000 s
16	> 60 s	0,001 s
32	> 60 s	0,002 s

Gli incontri sono sempre n/2, quindi "massimizza i pari" equivale a "minimizza i misti"

Per certificare l'ottimo del maximize, il solver deve escludere n/2+1 coppie: un ragionamento a **piccionaia**, difficilissimo per i solver CDCL

In ASP la formulazione conta quanto il modello.

Attività 2 · confronto_formulazioni.py, primo turno, timeout 60 s

CONCLUSIONI

Cosa portiamo a casa

Lezioni

A* vince dove ci sono trasposizioni; **IDA*** pareggia nei tempi dove non ce ne sono, con memoria lineare

Un'euristica migliore vale più di un algoritmo migliore

In ASP due formulazioni equivalenti possono differire di ordini di grandezza

Sviluppi possibili

IDA* con una tabella di trasposizione limitata

Linear conflict o pattern database per il puzzle

Euristica del labirinto che ignora le uscite irraggiungibili

Codice e dati: repository astar-to-wumpus · prolog-clingo/

SLIDE DI RISERVA · SE CHIEDONO DEL CUT

Un cut che sembra verde ma è rosso

```
dfs_cut(S, G, Soglia, _, _, _) :-
  euristica(S, H), F is G + H,
  F > Soglia, !,
  aggiorna_minimo_cut(F),
  fail.

aggiorna_minimo_cut(F) :-
  nb_getval(prossima_soglia, M),
  M \== inf, M =< F, !.
aggiorna_minimo_cut(F) :-
  nb_setval(prossima_soglia, F).
```

Il cut sembra servire solo a risparmiare un tentativo

Ma chi chiama esegue subito **fail**: il backtracking rientra e la seconda clausola **sovrascrive il minimo** con un f più grande

Senza il cut, su un grafo di 5 nodi IDA* trova costo **4 invece di 3**

ricerca/idastar_cut.pl · test/cut/demo_cut4.pl · test/cut/confronta_cut.py (27/27 identici a idastar.pl)